Deep Reinforcement Learning

Optimal Control & Feedback Control

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

Content

Content

- Recap of model-free RL
- Open-loop optimal control
  - Trajectory planning
  - Solution approaches
  - Stochastic optimal control
  - The linear case
- Closed-loop control
  - Model Predictive Control (MPC)
  - Differentiable Predictive Control (DPC)

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6-13	DQN, policy gradients, actor-critic, PPO/SAC, exploration	Deep RL from basics to modern algorithms
	Model-Based Control	
14	Optimal control & feedback control	How to do control when we know the model
	Model-based reinforcement learning	
	Advanced Topics	

Lecture contents

Recap: model-based and model-free RL

- Until now, knowledge of a model only in the simplest RL approaches.
  - Dynamic Programming for tabular RL, e.g., policy or value iteration.
  - We need to know $p(s'|s,a)$ to update our value estimate!
- All the advanced algorithms: **model-free RL**.
  - Collect experience by following the current policy π,
  - then update our estimate of V, Q, A or π only from the data.

Algorithm: Value iteration for estimating $\pi \approx \pi^*$.

while $(\Delta > \theta)$:
  $\Delta \leftarrow 0$
  for $s \in \mathcal{S}$:
    $V_{\text{old}}(s) \leftarrow V(s)$
    $V(s) \leftarrow \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p(s'|s,a)\left[r + \gamma V(s')\right]$
    $\Delta \leftarrow \max(\Delta, |V_{\text{old}}(s) - V(s)|)$

Advantages / disadvantages of model-free RL

+ No model knowledge required. All we need is access to the system / a black-box simulator.
+ No bias due to inaccurate model.
+ Easier to implement.
— Data-hungry! Can be very sample-inefficient.
— Ignorant of prior knowledge.

Today: Can we do better if we know the dynamics?

We often **know the dynamics**:

- Games (e.g., Chess, Go, Atari games).
- Easily modeled systems (e.g., car navigation).

Also, we can often **learn the dynamics**:

- System identification: fit unknown parameters of a known model.
- Learning: fit a general purpose model to observed transition data.

- Simulated environments (e.g., robotics, video games)

What is a model?

- Very generally, a **model** is an MDP tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$, just as we have seen before. Most notably, we have
  - the dynamics model $p\left(s'|s,a\right)$,
  - the reward probability $r \sim p\left(\cdot|s,a\right)$.
- The state and action spaces $\mathcal{S}$ and $\mathcal{A}$ are usually assumed to be known.
- The discount factor is given or can be chosen by an expert.

Types of models

1. Probabilistic vs. deterministic

- probabilistic:

$$s' \sim p\left(\cdot|s,a\right), \quad r \sim p\left(\cdot|s,a\right).$$

- deterministic (a special case of the former):

$$s' = f(s,a), \quad r = \ell(s,a).$$

2. Known vs. unknown

- **True MDP**: p and r (or f and ℓ) are perfectly known a priori.
- **Approximated MDP**: p and r (or f and ℓ) need to be learned,

$$p_\theta \approx p,\ r_\phi \approx r \qquad \text{or} \qquad f_\theta \approx f,\ \ell_\phi \approx \ell.$$

💡 The case of learning a model is covered extensively in the next lecture.

Open-loop optimal control

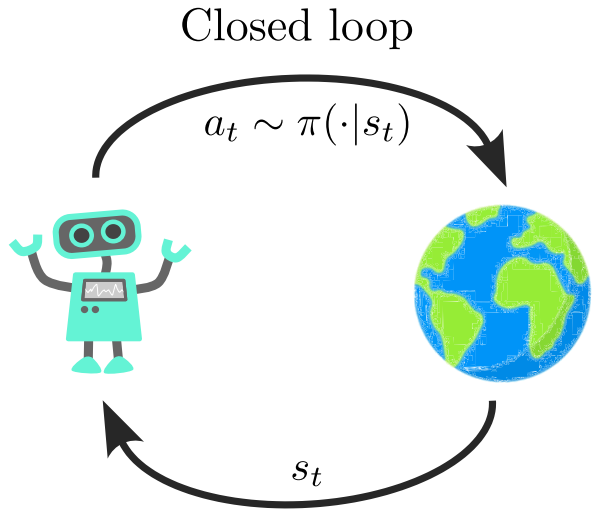
Planning & optimal control

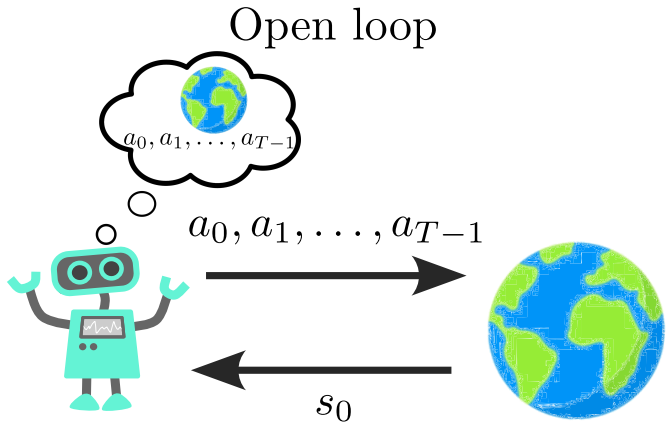
- For now, we asumme that the model
  - is known,
  - is fully deterministic.
- In the episodic case ($T < \infty$), the RL problem becomes

$$\min_{a_0,\dots,a_{T-1}} \sum_{t=0}^{T} \ell(s_t, a_t) = \min_{a_0,\dots,a_{T-1}} L(a) \quad (1)$$

$$\text{subject to} \quad s_{t+1} = f(s_t, a_t),\ t = 0, 1, \dots, T-1.$$

- Eq. (1) is referred to as an **optimal control problem** (**OCP**).
  - ℓ is the *stage cost*.
    - 💡 Closely related to the negative reward $-r$!
  - L is the *cost* that we seek to *minimize*.
    - 💡 Closely related to value V!
  - Optimal control is usually formulated as a *minimization problem*, e.g., the distance to a desired trajectory.
- Generally, optimal control refers to the **open-loop** / **planning** setting.





Only measure at s_0, no feedback

Inspired by Sergey Levine's CS285 lecture.

The deterministic vs. the stochastic case

- In RL, we usually assume that we have a stochastic problem for various reasons:
  - stochastic dynamics,
  - stochastic actions/policies,
  - stochastic rewards.
- In the open-loop setting, stochastic optimal control is suboptimal!
  $\Rightarrow$ Why?

Stochastic dynamics

$$p\left(s_0, \ldots, s_T | a_0, \ldots, a_{T-1}\right) = p(s_0) \prod_{t=0}^{T-1} p\left(s_{t+1} | s_t, a_t\right).$$

- The optimal action a_1 depends on s_1, which we cannot know for certain at $t = 0$.
- The same is true for all following states and actions.

Some approaches for stochastic optimal control

- Certainty Equivalence (CE)
  - Simply replace all stochastic quantities by their expectations/mean values.
    + Computationally cheap.
    − Completely ignores variance.
- Mean-variance / risk-sensitive optimization
  - define loss function composed of mean and variance:

$$\min_a \mathbb{E}\left[L\right] + \gamma \, \mathbf{var}\left[L\right].$$

    + Accounts for variance.
    − Need additional compute to estimate $\mathbf{var}\left[L\right]$.
- Sample-path optimization
  - Monte-Carlo estimate of expected loss by multiple simulations with different noise realizations ω_i:

$$\min_a \frac{1}{N} \sum_{i=1}^{N} L(a, \omega_i).$$

+ Can estimate arbitrary distributions (beyond Guassian).
− Compute grows with N.

Solving the OCP: single shooting

$$\min_{a_0,\dots,a_{T-1}} \underbrace{\sum_{t=0}^{T} \ell(s_t, a_t)}_{=L(a)} \quad \text{subject to} \quad s_{t+1} = f(s_t, a_t),\ t = 0, 1, \dots, T-1. \tag{1}$$

A very common approach to solve the OCP (1) is via **single shooting**, where assume that the states s_t are implicitly defined by choosing the actions a_t.

💡 This holds for most systems of interest. The basis for this are existence-and-uniqueness theorems, e.g., by Picard-Lindelöf.

Single shooting

Initialize: Initial state s_0, action sequence $a^{(0)} = \left(a_0^{(0)}, \dots, a_{T-1}^{(0)}\right)$.

for $j = 0, 1, 2, \dots$:

simulate the system following $a^{(j)}$ to get $s^{(j)} = \left(s_1^{(j)}, \dots, s_T^{(j)}\right)$

assess the performance by computing $L\left(a^{(j)}\right)$

gradient descent to update our actions:

$$a^{(j+1)} = a^{(j)} - \eta \nabla_a L\left(a^{(j)}\right)$$

Remaining question:

How do we compute the gradient

$$\nabla_a L\left(a^{(j)}\right)?$$

Gradients in single shooting (1)

$$\min_{a_0,\ldots,a_{T-1}} \underbrace{\sum_{t=0}^{T} \ell(s_t, a_t)}_{=L(a)} \quad \text{subject to} \quad s_{t+1} = f(s_t, a_t),\ t = 0, 1, \ldots, T-1. \tag{1}$$

The procedure of computing the gradient is closely related to the backpropagation algorithm!

1. Turn the constraint into an objective via the Lagrangian formalism, where λ is the **adjoint state**:

$$\begin{aligned}\mathcal{L}(s, a, \lambda) &= \sum_{t=0}^{T} \ell(s_t, a_t) + \sum_{t=0}^{T-1} \lambda_{t+1}^\top (f(s_t, a_t) - s_{t+1}) \\ &= \left[\ell(s_0, a_0) + \lambda_1^\top f(s_0, a_0)\right] + \left[\ell(s_1, a_1) + \lambda_2^\top f(s_1, a_1) - \lambda_1^\top s_1\right] + \ldots + \\ &\quad \left[\ell(s_t, a_t) + \lambda_{t+1}^\top f(s_t, a_t) - \lambda_t^\top s_t\right] + \ldots + \left[\ell(s_T, a_T) + \lambda_T^\top s_T\right].\end{aligned}$$

2. Necessary condition for optimality: Total derivative of $\mathcal{L}$ has to be zero:

$$\frac{\partial \mathcal{L}}{\partial \lambda_t} = 0, \quad \frac{\partial \mathcal{L}}{\partial s_t} = 0, \quad \frac{\partial \mathcal{L}}{\partial a_t} = 0.$$

Gradients in single shooting (2)

$$\mathcal{L}(s, a, \lambda) = \sum_{t=0}^{T} \ell(s_t, a_t) + \sum_{t=0}^{T-1} \lambda_{t+1}^\top (f(s_t, a_t) - s_{t+1})$$

3. State equation:

$$\frac{\partial \mathcal{L}}{\partial \lambda_t} = s_t - f(s_{t-1}, a_{t-1}) \stackrel{!}{=} 0 \quad \Leftrightarrow \quad s_{t+1} = f(s_t, a_t) \quad \text{for } t = 0, \dots, T-1.$$

4. Adjoint equation:

$$\frac{\partial \mathcal{L}}{\partial s_t} = \frac{\partial \ell}{\partial s_t} + \left(\frac{\partial f}{\partial s_t}\right)^\top \lambda_{t+1} - \lambda_t \stackrel{!}{=} 0 \quad \Leftrightarrow \quad \lambda_t = \frac{\partial \ell}{\partial s_t} + \left(\frac{\partial f}{\partial s_t}\right)^\top \lambda_{t+1} \quad \text{for } t = 0, \dots, T-1.$$

5. Final adjoint state ($t = T$): No influence on a future state, $\lambda_T = \frac{\partial \ell}{\partial s_T}$.

6. Action gradient:

$$\frac{\partial \mathcal{L}}{\partial a_t} = \frac{\partial \ell}{\partial a_t} + \left(\frac{\partial f}{\partial a_t}\right)^\top \lambda_{t+1} \quad \text{for } t = 0, \dots, T-1.$$

Gradients in single shooting (3)

Gradient calculation in the single shooting method for solving (1)

Given: Initial state s_0, action sequence $a = (a_0, \dots, a_{T-1})$.

- **Forward simulation** of the state equation:

$$s_{t+1} = f(s_t, a_t), \qquad t = 0, 1, \dots, T-1.$$

- "Initial condition" (i.e., at final time T) for the adjoint equation: $\lambda_T = \frac{\partial \ell}{\partial s_T}$.
- **Backward simulation** of the adjoint equation:

$$\lambda_t = \frac{\partial \ell}{\partial s_t} + \left(\frac{\partial f}{\partial s_t}\right)^\top \lambda_{t+1}, \qquad t = T-1, T-2, \dots, 1, 0.$$

- **Gradient calculation** w.r.t. a:

$$\frac{\partial \mathcal{L}}{\partial a_t} = \frac{\partial \ell}{\partial a_t} + \left(\frac{\partial f}{\partial a_t}\right)^\top \lambda_{t+1}, \qquad t = 0, 1, \dots, T-1.$$

Notes

- The backward simulation is closely related to the backward pass in backpropagation.
- In modern packages such as PyTorch, JAX or TensorFlow, this procedure can be automated with autodiff, when appropriately implementing the forward pass.

Solving the OCP: full discretization

- Alternative to forward-backward: the full discretization
- Both $\boldsymbol{s}$ and $\boldsymbol{a}$ become optimization variables:

$$x = \left[s_0^\top, a_0^\top, s_1^\top, a_1^\top, \ldots, a_{T-1}^\top, s_T^\top\right]^\top.$$

- Reformulation of the OCP (1)

$$\min_x F(x) = \sum_{t=0}^{T} \ell(s_t, a_t) \quad \text{s.t.} \quad C(x) = \begin{bmatrix} f(s_0, a_0) - s_1 \\ f(s_1, a_1) - s_2 \\ \vdots \\ f(s_{T-1}, a_{T-1}) - s_T \end{bmatrix} = 0 \tag{2}$$

- Total number of optimization variables when the state dimension is n and the action dimension is m: $(T+1)n + Tm$.
- Solution via standard solvers for nonlinear, constrained optimization problems:
  - Requires the gradients $\nabla_x F(x)$ and $\nabla_x C(x)$.
  - The latter is a very large matrix, but it is fortunately very sparse.

Comparison: Shooting vs. full discretization

	Shooting (1)	**Full discretization (2)**
Sensitivity to initialization	Very **sensitive to poor initial guesses** $a^{(0)}$.	Because we initialize the entire state trajectory s, the system doesn't have to be dynamically feasible during early solver iterations $\Rightarrow$ Significantly **more stable for unstable dynamics**.
Computational cost vs. convergence	**Fast iterations** (one forward and backward pass), but may take **many iterations** to converge.	**Iterations more expensive**, but standard Sequential Quadratic Programming (SQP) or Interior Point methods achieve quadratic convergence near the solution, requiring far **fewer total iterations**.
Constraint Handling	Handling state constraints requires **challenging** penalty functions or barrier methods.	Limits on states like $s_{\text{min}} \leq s_t \leq s_{\text{max}}$ or actuations ($a_{\text{min}} \leq a_t \leq a_{\text{max}}$) **can be handled easily**. They are simply treated as bound constraints on elements of our vector x.

The linear-quadratic case

The linear-quadratic case

Let's consider a special (yet very common) case

- **Linear dynamics**:

$$s_{t+1} = As_t + Ba_t, \quad \text{with } A \in \mathbb{R}^{n\times n} \text{ and } B \in \mathbb{R}^{n\times m}.$$

- **Quadratic losses**:

$$L(s,a) = \left(\sum_{t=0}^{T-1} s_t^\top Q s_t + a_t^\top R a_t\right) + s_T^\top Q_f s_T,$$

with matrices $Q, Q_f \in \mathbb{R}^{n\times n}$ penalizing the (terminal) state and $R \in \mathbb{R}^{m\times m}$ penalizing the control cost.

Additional conditions

- The matrix Q is *positive semidefinite*:

$$s^\top Q s \geq 0 \; \forall \, s \in \mathbb{R}^n.$$

- The matrix R is *positive definite*:

$$a^\top R a > 0 \; \forall \, a \in \mathbb{R}^m.$$

- These ensure existence and uniquenes of a minimizer s^*, a^*.

Linear-quadratic optimal control problem

$$\begin{aligned} &\min_{s,a} \left(\sum_{t=0}^{T-1} s_t^\top Q s_t + a_t^\top R a_t\right) + s_T^\top Q_f s_T \\ \text{subject to} \quad &s_{t+1} = As_t + Ba_t, \; t = 0, 1, \ldots, T-1. \end{aligned} \tag{3}$$

Removing the constraints

Let's explicitly calculate $s_1, s_2, \ldots$:

$$s_1 = As_0 + Ba_0$$
$$s_2 = As_1 + Ba_1 = A(As_0 + Ba_0) + Ba_1 = A^2 s_0 + ABa_0 + Ba_1$$
$$s_3 = As_2 + Ba_2 = A(A^2 s_0 + ABa_0 + Ba_1) + Ba_2 = A^3 s_0 + A^2 Ba_0 + ABa_1 + Ba_2$$

General rule: $s_t = A^t s_0 + \sum_{i=0}^{t} A^{t-i-1} B a_i$

Explicitly:

$$\underbrace{\begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ \vdots \\ s_T \end{bmatrix}}_{\hat{s} \in \mathbb{R}^{(T+1)n}} = \underbrace{\begin{bmatrix} 0 & 0 & 0 & \cdots & B \\ 0 & 0 & 0 & \cdots & 0 \\ AB & B & 0 & \cdots & 0 \\ A^2B & AB & B & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A^{T-1}B & A^{T-2}B & A^{T-3}B & \cdots & B \end{bmatrix}}_{G \in \mathbb{R}^{(T+1)n \times Tm}} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{T-1} \end{bmatrix}}_{\hat{a} \in \mathbb{R}^{Tm}} + \underbrace{\begin{bmatrix} I \\ A \\ A^2 \\ A^3 \\ \vdots \\ A^T \end{bmatrix}}_{H \in \mathbb{R}^{(T+1)n}} s_0$$

In short: $\hat{s} = G\hat{a} + Hs_0$

Reformulating the cost function

- Next, we stack our penalty matrices such that they match the dimensions of S and U:

$$\hat{Q} = \begin{bmatrix} Q & & & \\ & \ddots & & \\ & & Q & \\ & & & Q_f \end{bmatrix} \in \mathbb{R}^{(T+1)n \times (T+1)n}, \qquad \hat{R} = \begin{bmatrix} R & & \\ & \ddots & \\ & & R \end{bmatrix} \in \mathbb{R}^{\mathrm{T}m \times Tm}.$$

- We can now insert this into the linear-quadratic OCP (3):

$$\min_{s,a} \left(\sum_{t=0}^{T-1} s_t^\top Q s_t + a_t^\top R a_t \right) + s_T^\top Q_f s_T \quad \text{s.t.} \quad s_{t+1} = A s_t + B a_t,\ t = 0, 1, \ldots, T-1$$

becomes

$$\min_{\hat{s},\hat{a}} \hat{s}^\top \hat{Q} \hat{s} + \hat{a}^\top \hat{R} \hat{a} \quad \text{s.t.} \quad \hat{s} = G\hat{a} + H s_0.$$

- **Question**: Do we have to consider $\hat{s}$ and $\hat{a}$ independently?
  $\Rightarrow$ Let's simply insert the constraints!

$$\boxed{\min_{\hat{a}} \left(G\hat{a} + H s_0\right)^\top \hat{Q} \left(G\hat{a} + H s_0\right) + \hat{a}^\top \hat{R} \hat{a}} \tag{4}$$

Solving the linear OCP

$$
\begin{aligned}
&\min_{\hat{a}} \left(G\hat{a}+Hs_0\right)^\top \hat{Q}\left(G\hat{a}+Hs_0\right) + \hat{a}^\top \hat{R}\hat{a} \\
=&\min_{\hat{a}} \left(\hat{a}^\top G^\top + s_0{}^\top H^\top\right)\hat{Q}\left(G\hat{a}+Hs_0\right) + \hat{a}^\top \hat{R}\hat{a} \\
=&\min_{\hat{a}} \hat{a}^\top G^\top \hat{Q} G\hat{a} + 2\hat{a}^\top G^\top \hat{Q} H s_0 + s_0{}^\top H^\top \hat{Q} H s_0 + \hat{a}^\top \hat{R}\hat{a} \\
=&\min_{\hat{a}} \hat{a}^\top \left(G^\top \hat{Q} G + \hat{R}\right)\hat{a} + 2\hat{a}^\top G^\top \hat{Q} H s_0 + s_0{}^\top H^\top \hat{Q} H s_0
\end{aligned}
$$

Some linear algebra

$$(G\hat{a})^\top = \hat{a}^\top G^\top$$
$$s_0^\top H^\top \hat{Q} G\hat{a} = \hat{a}^\top G^\top \hat{Q} H s_0$$

- This is a quadratic polynomial in $\hat{a}$!
- Since Q and R (and thus, $\hat{Q}$ and $\hat{R}$) are positive (semi-)definite, the loss function is strictly convex $\Rightarrow$ unique minimizer.
- Take the drivative with respect to $\hat{a}$:

$$\frac{\mathrm{d}}{\mathrm{d}\hat{a}}\left(\hat{a}^\top \left(G^\top \hat{Q} G + \hat{R}\right)\hat{a} + 2\hat{a}^\top G^\top \hat{Q} H s_0 + s_0{}^\top H^\top \hat{Q} H s_0\right) = 2\left(G^\top \hat{Q} G + \hat{R}\right)\hat{a} + 2G^\top \hat{Q} H s_0.$$

- Setting the gradient to zero yields a linear system (depending on the initial condition s_0):

$$\boxed{\left(G^\top \hat{Q} G + \hat{R}\right)\hat{a} = -G^\top \hat{Q} H s_0.} \tag{5}$$

Example: Control of a forklift (1)

- Consider a simple vehicle driving on a plane (with mass m and friction d):

$$s = \begin{bmatrix} p_1 \\ v_1 \\ p_2 \\ v_2 \end{bmatrix}, \quad \dot{s} = \frac{\mathrm{d}s}{\mathrm{d}t}(t) = A_c s(t) + B_c a(t) = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -d/m & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -d/m \end{bmatrix} s(t) + \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} a(t).$$



- This can be translated into a discrete-time system with time step Δt:

$$A = \exp(A_c \Delta t), \quad B = \left(\int_0^{\Delta t} \exp(A(\Delta t - \tau)) \, \mathrm{d}\tau \right) B_c \quad \Longrightarrow \quad s_{t+1} = A s_t + B a_t.$$

- Control objective: Steer the forklift from $s_0 = [2\ 0\ 0\ 0]^\top$ to the origin with a small penalty on the control:

$$\min_{s,a} \sum_{t=0}^{T} \|s_t\|_2^2 + \alpha \|a_t\|_2^2 \quad \text{s.t.} \quad s_{t+1} = A s_t + B a_t.$$

Example: Control of a forklift (2)

- Define individual control matrices:

$$Q = \begin{bmatrix} 1 & & & \\ & 0 & & \\ & & 1 & \\ & & & 0 \end{bmatrix}, \quad Q_f = 10 \cdot Q, \quad R = \begin{bmatrix} \alpha & \\ & \alpha \end{bmatrix}.$$

- Assemble the matrices $\hat{Q}$ and $\hat{R}$. With $T = 200$, we get

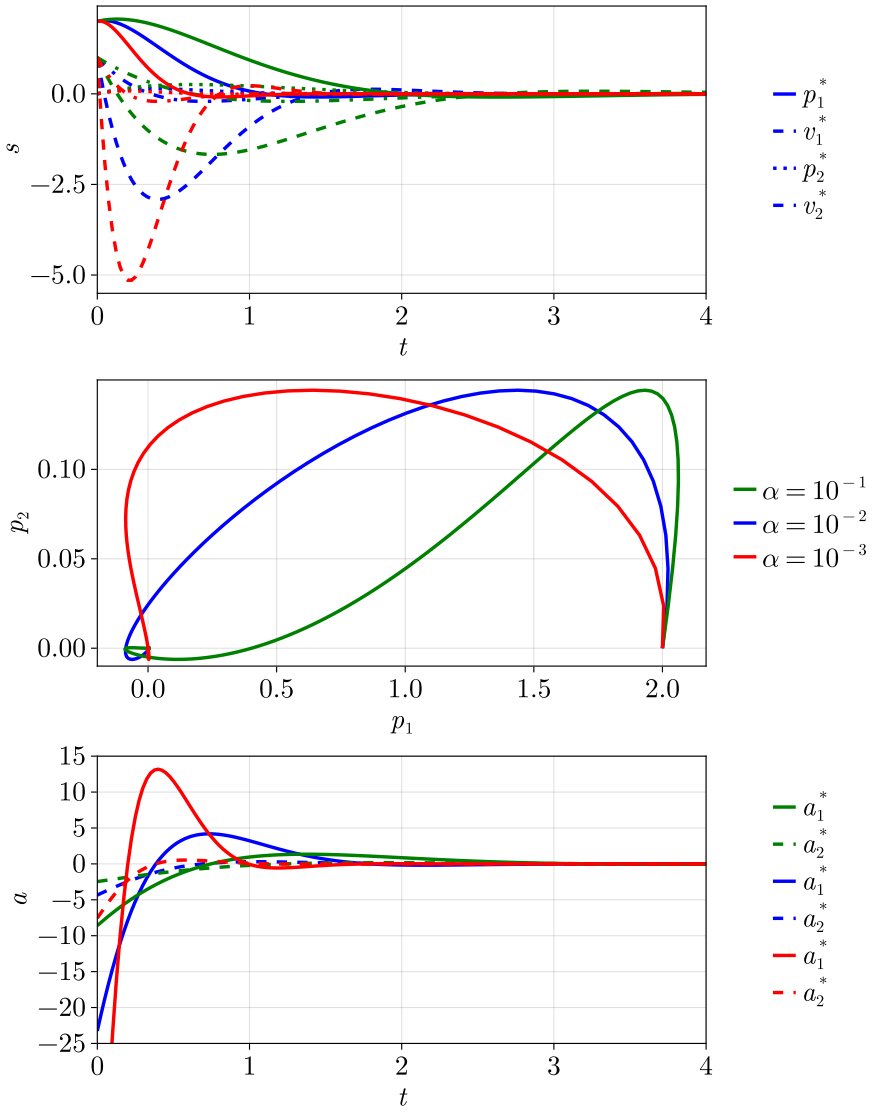
$$\hat{Q} \in \mathbb{R}^{804 \times 804} \quad \text{and} \quad \hat{R} \in \mathbb{R}^{400 \times 400}.$$

- Given the initial condition s_0, solve the linear system

$$\left(G^\top \hat{Q} G + \hat{R}\right) \hat{a} = -G^\top \hat{Q} H s_0. \qquad (5)$$

- Simulate dynamics to get optimal trajectory s^*:

$$s^*_{t+1} = A s^*_t + B a^*_t \qquad \text{or} \qquad \hat{s}^* = G \hat{a}^* + H s_0.$$



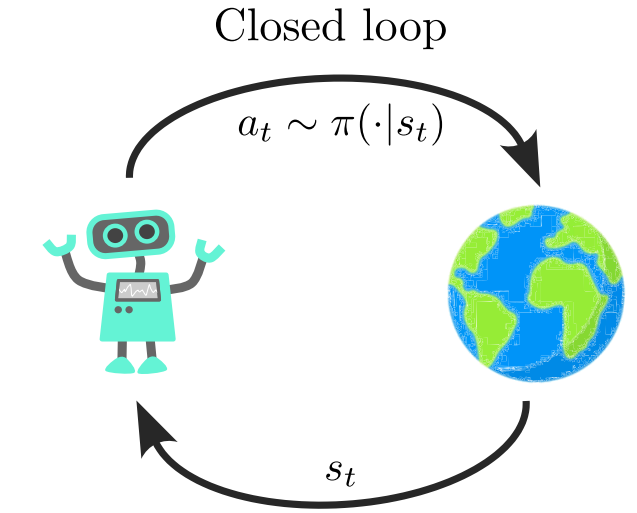
Closed-loop control

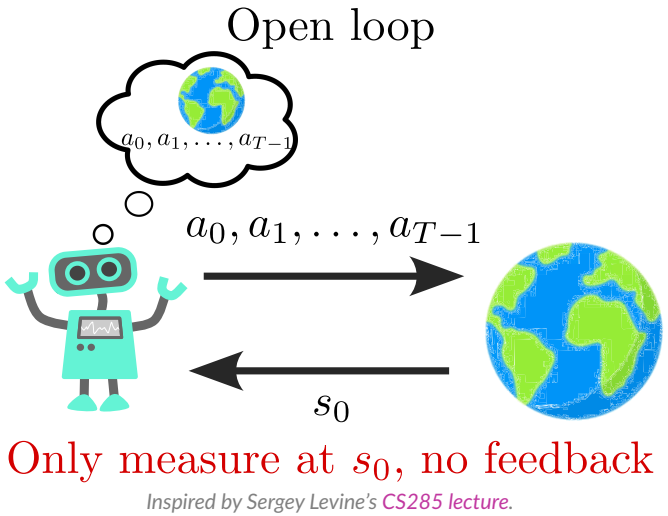
Model predicitve control (MPC)

- Given a model, solving open-loop OCPs is a well-established field.
- But there are also **drawbacks** in the application to real systems.
  - Unforeseen events cannot be accounted for.
  - External disturbances/uncertainties are hard/impossible to include.
  - Smalles inaccuracies scale exponentially over time.
- **Question**: Can we close the loop around OCPs?

Model predictive control (MPC)

- Let us solve the open-loop problem repeatedly on a shorter horizon $p \leq T$.
- The “feedback” is setting the initial condition of the OCP.

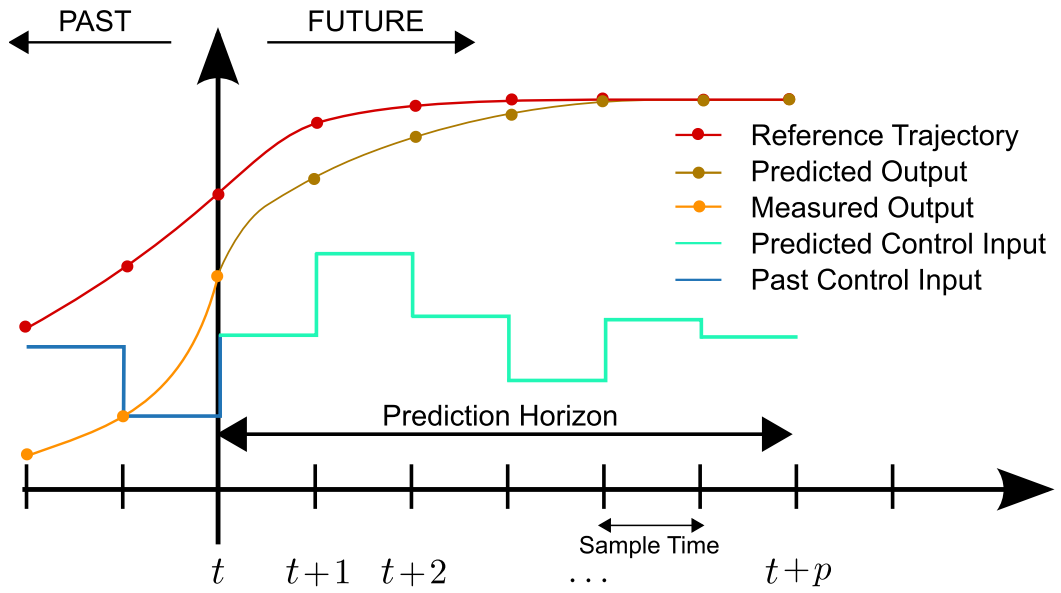




Inspired by Sergey Levine's CS285 lecture.

MPC scheme

$$\text{OCP:} \quad \min_{a_0,\dots,a_{T-1}} \sum_{t=0}^{T} \ell(s_t, a_t) = \min_{a_0,\dots,a_{T-1}} L(a) \quad \text{s.t.} \quad s_{t+1} = f(s_t, a_t),\ t = 0, 1, \dots, T-1. \tag{1}$$



Source: Wikipedia.

The MPC loop

1. Measure system state s_t and use it as the initial state for the OCP (1).
2. Solve (1) using a model of the real system over a prediction horzion of length p.
3. Apply first entry of a^* (i.e., a_0^*) to the real system.
4. Advance real system by one time step.
5. Repeat.

Pros and cons

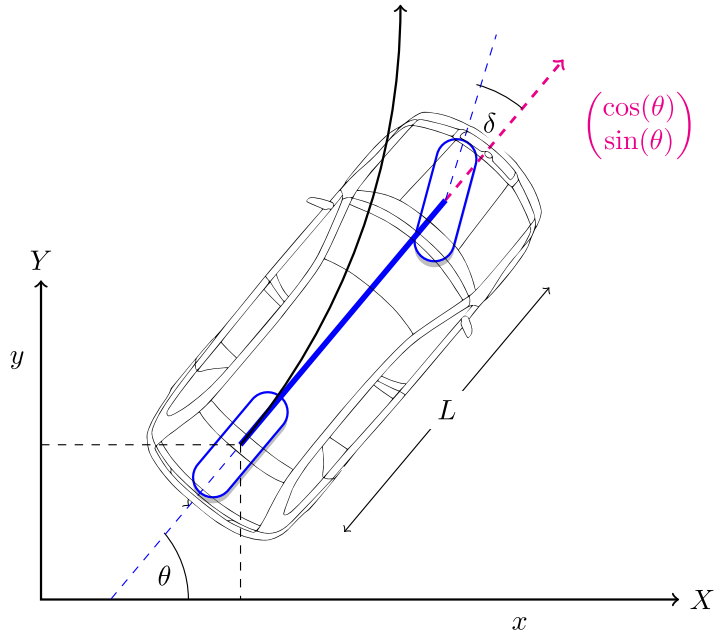
+ Very robust.
+ No offline learning phase (if model known).
+ Easy to include state constraints.
+ Lots of theory. (e.g., (Grüne and Pannek 2017))

— Real-time capability (in particular for nonlinear models).
— Control performance depends on model quality.

Example: Autonomous driving

We want to plan the trajectory of an autonomous vehicle from an initial position towards a terminal position.

- Dynamics: Simplified **kinematic bicycle model** (Kong et al. 2015).



Source: *Algorithms for Automated Driving.*

$$s = \begin{bmatrix} x \\ y \\ \theta \\ v \end{bmatrix}, \; a = \begin{bmatrix} \text{acc} \\ \delta \end{bmatrix},$$

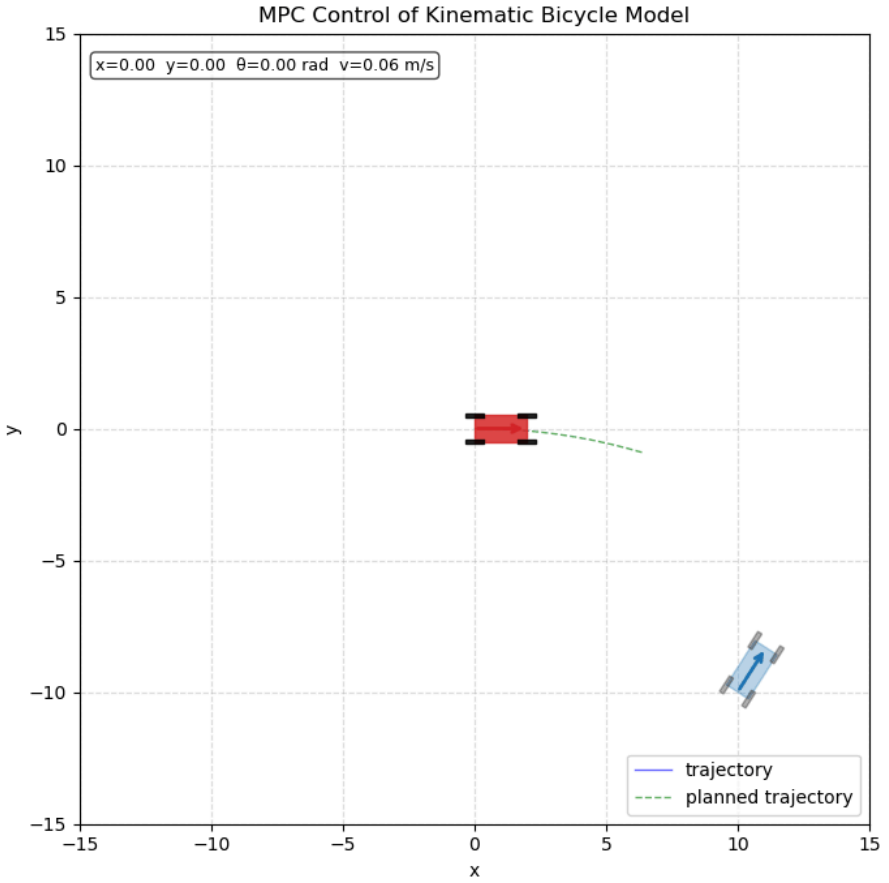
$$\dot{s} = \begin{bmatrix} s_4 \cos s_3 \\ s_4 \sin s_3 \\ \frac{s_4}{L} \tan a_2 \\ a_1 - \lambda s_4 \end{bmatrix}.$$

- Objective: **terminal condition** to approach target location

$$\min_{a_0,\ldots,a_{p-1}} \ell(s_p) = \left\| s_p - s_p^{\text{target}} \right\|_2^2 \quad \text{s.t.} \quad \textbf{bicycle model.}$$

- Discretization: $\Delta t = 0.2, p = 25.$

- MPC optimization: Adam ($\eta = 0.1$); 10 steps.
- Results:



Using linearized models in MPC

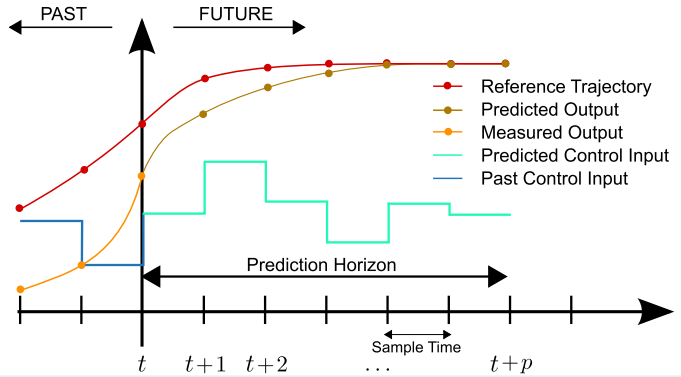
- Approach to avoid the large online cost in many MPC applications: use **linearized dynamics** instead!
  - 💡 This is a very common approach in control theory, in particular linearization around the desired state s^{target}.
- Approach via Taylor series expansion around current state $\bar{s}_k, \bar{a}_k$.
- Introduce distance to current state:

$$\Delta s_t = s_t - \bar{s}_t, \quad \Delta a_t = a_t - \bar{a}_t.$$

- First-order (i.e., **linear**) approximation in Δs_t and Δa_t:

$$s_{t+1} \approx \underbrace{f(\bar{s}_t, \bar{a}_t)}_{=\bar{s}_{t+1}} + \underbrace{\left.\frac{\partial f}{\partial s}\right|_{(\bar{s}_t, \bar{a}_t)}}_{=A} \Delta s_t + \underbrace{\left.\frac{\partial f}{\partial a}\right|_{(\bar{s}_t, \bar{a}_t)}}_{=B} \Delta a_t + \mathcal{O}(\Delta s_t^2, \Delta a_t^2)$$

$$s_{t+1} - \bar{s}_{t+1} = \boxed{\Delta s_{t+1} \approx A \Delta s_t + B \Delta a_t.} \qquad (6)$$



MPC with linearized dynamics
for $t = 0, 1, 2, \ldots$:
- **Measure** the current state $\bar{s}_t, \bar{a}_t$ of the real (nonlinear system).
- **Linearize** the system dynamics around $\bar{s}_t, \bar{a}_t \Rightarrow$ (6).
- **Assemble** matrices $\hat{Q}_t$ / $\hat{R}_t$ and G_t / H_t.
- **Solve** linear problem (4).
- **Apply** first entry to real system.

New bottlenecks: ● The linearization around $\bar{s}_t, \bar{a}_t$ ● Assembly of $\hat{Q}_t$ / $\hat{R}_t$ and G_t / H_t.

Relation to reinforcement learning

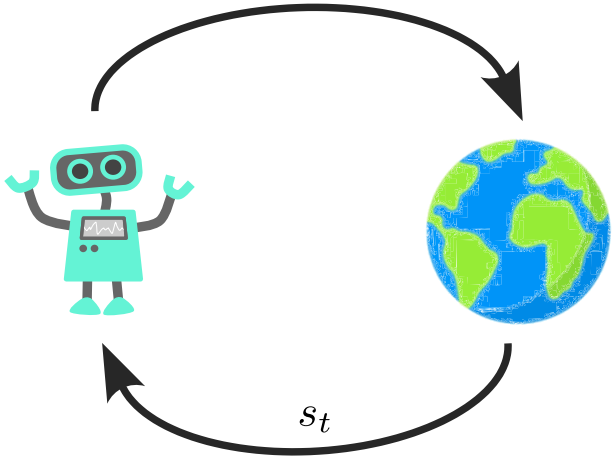
Reinforcement Learning

$$a_t \sim \pi(\cdot|s_t)$$

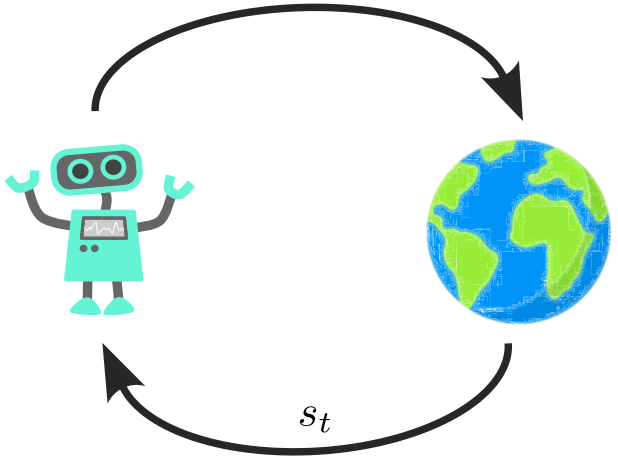
vs.

Model Predictive Control

$$a_t = \arg\min_{\tilde{a}} \sum_{\tau=0}^{p} \ell(\tilde{s}_\tau, \tilde{a}_\tau)$$
$$\text{s.t. } \tilde{s}_{\tau+1} = f(\tilde{s}_\tau, \tilde{a}_\tau)$$
$$\tilde{s}_0 = s_t$$



vs.



Trade-offs

- Learning a policy offline vs. solving an OCP online.
- Expensive training vs. expensive inference.

Approaches in between

- Explicit MPC.
- Differentiable predictive control (DPC).
- Monte-Carlo tree search (MCTS).

Explicit MPC

- MPC's main challenge: **large online cost**!
- Can we avoid this? ⇒ Let's revisit the OCP formulation (1) in the MPC setting:

$$\min_{\tilde{a}} \sum_{\tau=0}^{p} \ell(\tilde{s}_\tau, \tilde{a}_\tau) \qquad (7)$$
$$\text{s.t.} \quad \tilde{s}_{\tau+1} = f(\tilde{s}_\tau, \tilde{a}_\tau),\ \tilde{s}_0 = s_t.$$

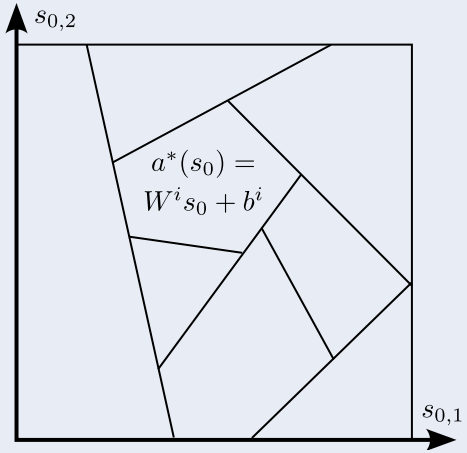
- This OCP is parameterized by the initial condition s_0!
- If we solve it for every $s_0 \in \mathcal{S}$ and explicitly store $a^*(s_0)$ in a library, we can quickly retrieve it in the online phase.
- But there are infinitely many s_0! 😱
- Options:
  - Approximate by a finite number of s_0.
  - Special case: linearity.

The explicit linear-quadratic regulator (Bemporad et al. 2002)

- For linear models and quadratic costs,

$$f(s,a) = As + Ba,$$
$$\ell(s,a) = s^\top Q s + a^\top R a,$$

the solution space $a^*(s_0)$ for (7) is split into polygons on which the optimal feedback a^* is an affine function of the initial condition s_0



- Finite number of polygons ⇒ finite number of OCPs to solve!
- Algorithm for the **offline phase**:
  1. Split the space of initial conditions into polygons $i = 1, \ldots, N_p$.
  2. Determine coefficients W^i and b_i for the affine feedback law (details in (Bemporad et al. 2002)).
- Drawback: Exponential growth of polygons with state dimension n.
  - Workaround: Replace polygon by deep neural net (Karg and Lucia 2020).

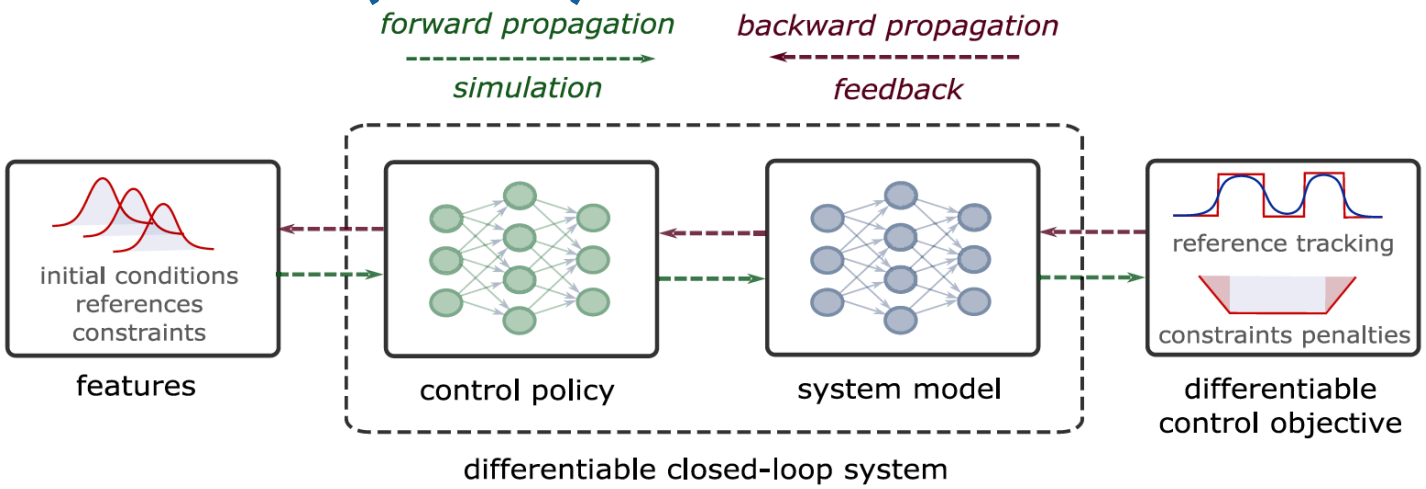
Differentiable predictive control (DPC)

- Let's start with a policy network, just as in policy gradient methods: $a = \mu_\phi(s)$.
- Inserting into the right-hand side of our model yields an autonomous system:

$$s_{t+1} = f(s_t, \mu_\phi(s_t)).$$

- We can now formulate a closed-loop performance criterion:

$$L(\phi) = \sum_{t=1}^{T} \left\| s_t - s_t^{\text{target}} \right\| + \alpha \left\| \mu_\phi(s_t) \right\|.$$



Differentiable predictive control framework (Drgoňa et al. 2022).

- If the dynamics model f (or a learned version f_θ; see next lecture) is differentiable, then we can directly optimize the policy paramter ϕ via backpropagation and gradient descent:

$$\phi \leftarrow \phi - \eta \frac{\mathrm{d}L}{\mathrm{d}\phi}.$$

- This is known as **differentiable predictive control** (**DPC**).

Summary / what we have learned

- Knowing a model of the system dynamics can be very useful, not only for tabular reinforcement learning methods.
- Open-loop = Planning: Given an initial condition s_0, find the optimal actions for an entire trajectory **once**.
- MPC: Repeated solution of open-loop optimal control problems.
- Explicit MPC: Solving the open-loop problem in an offline phase in a parameterized fashion.
- Differentiable predictive control (DPC): Policy optimization via direct differentiation through the dynamics model.

References

Bemporad, Alberto, Manfred Morari, Vivek Dua, and Efstratios N. Pistikopoulos. 2002. “The Explicit Linear Quadratic Regulator for Constrained Systems.” *Automatica* 38 (1): 3–20. doi:https://doi.org/10.1016/S0005-1098(01)00174-1.

Drgoňa, Ján, Karol Kiš, Aaron Tuor, Draguna Vrabie, and Martin Klaučo. 2022. “Differentiable Predictive Control: Deep Learning Alternative to Explicit Model Predictive Control for Unknown Nonlinear Systems.” *Journal of Process Control* 116: 80–92.

Grüne, L., and J. Pannek. 2017. *Nonlinear Model Predictive Control*. 2nd ed. Springer International Publishing.

Karg, Benjamin, and Sergio Lucia. 2020. “Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning.” *IEEE Transactions on Cybernetics* 50 (9): 3866–78. doi:10.1109/TCYB.2020.2999556.

Kong, Jason, Mark Pfeiffer, Georg Schildbach, and Francesco Borrelli. 2015. “Kinematic and Dynamic Vehicle Models for Autonomous Driving Control Design.” In *2015 IEEE Intelligent Vehicles Symposium (IV)*, 1094–99. doi:10.1109/IVS.2015.7225830.